

- Combinational Logic
- Sequential Logic
- Custom Single-Purpose Processor Design
- Optimizing Custom Single-Purpose Processors

2.1 Combination Logic

Combinational circuit is a digital circuit whose output is purely a function of its present inputs. Combination logic circuits are made up from basic gates or universal gates that are combined or connected together to produce more complex switching circuits. In general, logic gates are the building blocks of combinational logic circuits. It has no memory block. Some of the examples of the combinational circuits are decoder, multiplexer, adder, ROM etc.

CMOS Transistors

A transistor, which acts as a simple on/off switch, is the basic electrical component in digital system. More abstract components, logic gates, are formed with the combinations of transistors. In Complementary Metal Oxide Semiconductor (CMOS), the gate voltage controls the flow of current from source to drain. The nMOS conducts when gate is at high voltage (5v) whereas pMOS conducts when gate is at low voltage (0v). The symbol for nMOS and pMOS is shown in the figure 2.1.

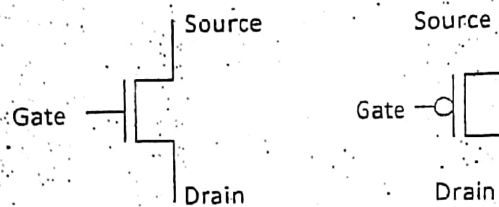


Figure 2.1: nMOS and pMOS transistors

Different gates and boolean functions can be realized using nMOS and pMOS.

- A. **Inverter:** When $x = 0$, transistor T1 conducts but T2 does not. So, output is logic 1. And when $x = 1$, T2 conducts but T1 does not.

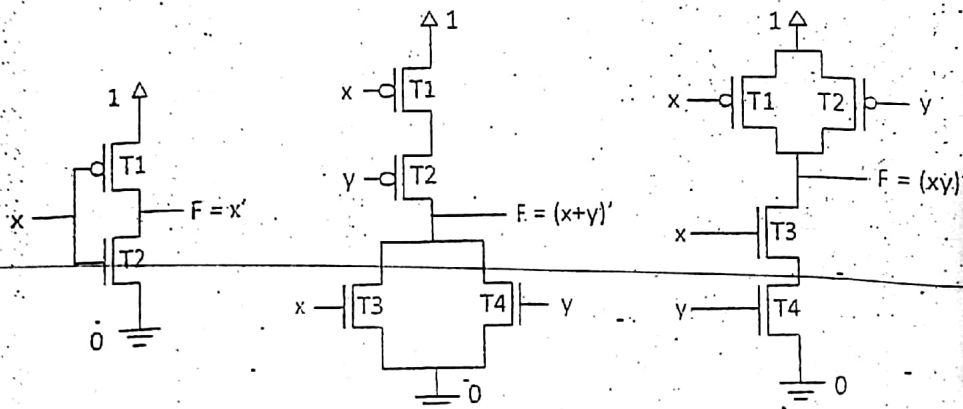


Figure 2.2: Inverter, NOR and NAND gate using nMOS and pMOS

B. NOR Gate

When $x = 0$ and $y = 0$, then T1 and T2 conduct but T3 and T4 don't. So, F is connected to Vcc.

When $x = 1$ and $y = 0$, then T2 and T3 conduct but T1 and T4 don't. So, F is connected to ground.

When $x = 0$ and $y = 1$, then T1 and T4 conduct but T2 and T3 don't. So, F is connected to ground.

When $x = 1$ and $y = 1$, then T3 and T4 conduct but T1 and T2 don't. So, F is connected to ground.

When at least one of the two inputs is high then the output is connected to ground. And when both inputs are low then the output is connected to Vcc.

C. NAND Gate

When $x = 0$ and $y = 0$, then T1 and T2 conduct but T3 and T4 don't. So, F is connected to Vcc.

When $x = 1$ and $y = 0$, then T2 and T3 conduct but T1 and T4 don't. So, F is connected to Vcc.

When $x = 0$ and $y = 1$, then T1 and T4 conduct but T2 and T3 don't. So, F is connected to Vcc.

When $x = 1$ and $y = 1$, then T3 and T4 conduct but T1 and T2 don't. So, F is connected to ground.

When at least one out of two inputs is low then the output is connected to Vcc. And when both inputs are high then the output is connected to ground.

Basic Logic Gates

- The NOT (Inverter) gate simply complements the input.
- The AND gate outputs 1 if and only if all of its inputs are 1.
- The OR gate outputs 1 if at least one of its inputs is 1.
- The XOR (exclusive-OR) gate outputs 1 when only one of its inputs is 1.
- The NAND, NOR and XNOR gates outputs the complement of AND, OR and XOR respectively.

Basic Combinational Logic Design

In Combinational Design, output is purely a function of its present inputs and has no memory of past inputs. We can use basic logic gates to design combinational circuits. In such design, outputs are described in terms of inputs.

General steps for combinational Logic Design

- The description is translated into a truth table with all possible combinations of input values.
- The input values lies on the left of the truth table and the corresponding output values of the inputs lies on the right of the truth table.
- For each output, we have to derive the equations. The equation may contain number of combinations of the inputs. The number of combinations depends on the number of high (1)

value on each column of the output. Rows of the inputs are used to derive the equation corresponding to the high output of the column. And the equation must be further minimized.

- Another way to derive minimized equation directly is by using k-map. It is always better to use k-map unless the design is too simple (when the output column consists of only one high value).
- The final equation is translated to an equivalent circuit diagram using logic gates.

Combinational Logic Design Example

Example 1: In an alarm system of a bank, three sensors are implemented and the alarm is triggered when at least two sensors detect the change. Assuming sensors to output digital values, design a combinational logic circuit for alarm system.

Solution: Let a, b, c represent the three sensors and y represents the buzzer for alarm. The output y should be high when two or more than two inputs are high. The truth table and its corresponding combinational design are shown below.

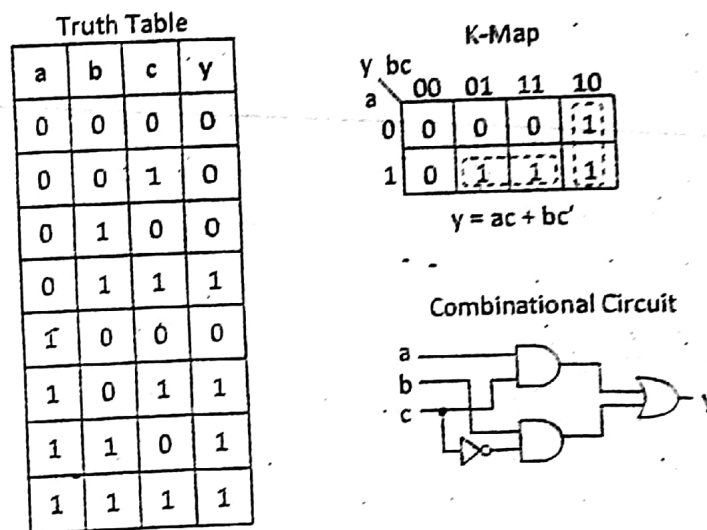


Figure 2.3: Truth table, K-map, and combinational circuit for bank alarm system

RT-Level Combinational Components

Register-transfer or RT level components are generally used when the design of the circuit becomes complex. As the number of input increase, the complexity of the design increase. One of the ways to reduce design complexities is by using RT-level components. Multiplexers, decoder, adder are the examples of RT-Level Components.

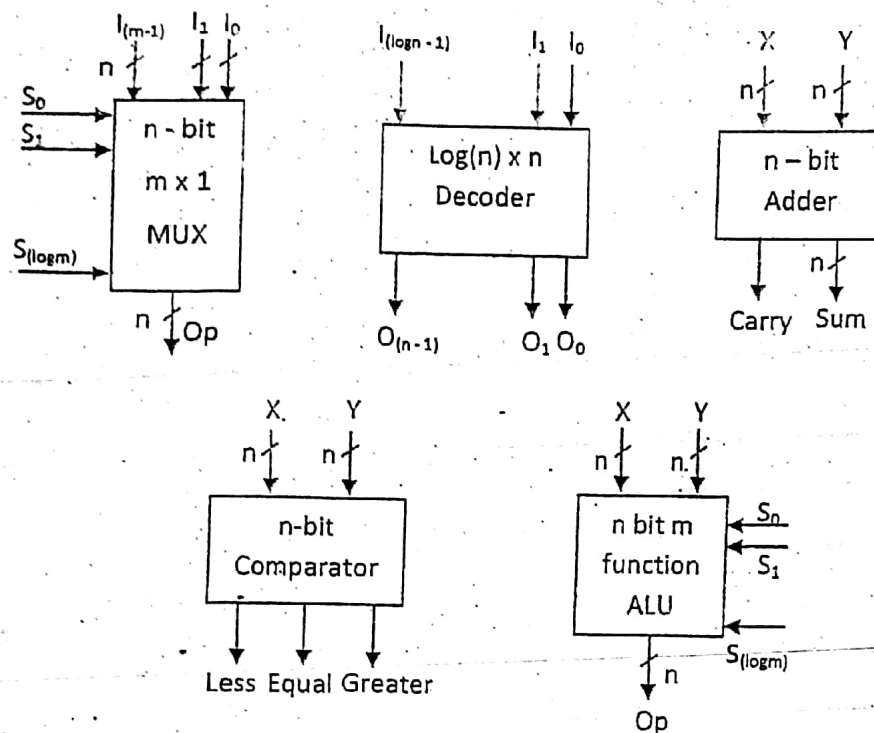


Figure 2.4: Few commonly used RT-Level Combinational Components

- A multiplexer allows only one of its data inputs to pass through to the output. For $m \times 1$ multiplexer there are m data inputs and one data output with $\log_2 m$ select lines. The value of select line determines which input data to pass through to the output. It can be used for parallel to serial conversion.
- A decoder allows exactly one of the output lines to be high at a given time for a particular input. For n input lines there will be 2^n output lines. A decoder can be used for coding the addressing lines in the memory. It can be used to convert binary to a suitable form.
- An adder is used to add two n -bit inputs producing an n -bit sum along with a carry of 1 bit.
- A comparator allows to compare two n -bit binary inputs, generating the corresponding output based on whether one input is less than, equal to, or greater than another input.
- An arithmetic-logic unit (ALU) performs variety of arithmetic and logic functions on its n -bit inputs. The select line is used to select which function is to be carried out. If there are 2^m functions that can be done by ALU then there must be at least m select lines.
- A shifter is another example which is used to shift the bits of the input right or left. It can be used as a divider or multiplier. For example shifting 0110 (6) to the right would give 0011 (3).

2.2 Sequential Logic

A sequential circuit is a digital circuit whose outputs are a function of not only the present inputs but also the past inputs. The output of a sequential logic depends on its present internal state and the present inputs. Hence a sequential logic circuit has some kind of memory. Logic gates and flip flops are the basic building blocks of sequential logic circuits. Flip flop is an example of sequential logic circuit.

A flip flop stores a single bit. The different types of flip flops are listed below.

- **D-flip flop:** It has two inputs D and clock, when clock is high, value of D is stored in flip flop and same will be the value of the output Q. When clock is low, previously stored bit is maintained ignoring the value of input D.
- **SR flip flop:** It has three inputs S (set), R (reset) and clock. When clock is low, the previously stored bit is maintained ignoring the values of input at S and R. When clock is high, the output varies with inputs S and R. If S is high, the output Q will be high and high bit (1) will be stored by the flip flop. If R is high, then low bit (0) will be stored. The output will not change if both the inputs are low but the undefined condition will occur if both the inputs are high.
- **JK flip flop:** Its operation is similar to that of SR flip flop but when both the inputs J and K is high, the stored bit toggles either from high to low or low to high.

Flip flops are generally designed to be edge triggered to prevent the unexpected behavior from signal glitches, the inputs are checked either at the rising edge or falling edge of the clock. Glitches represent an undesirable transition that occurs before the signal settles to its intended value.

RT Level Sequential Components

Generally RT level sequential components are required for designing complex sequential systems. Counters and registers are examples of RT level sequential components.

- A **register** stores n bits from its n bit data input which also appears at its output. A register usually has at least two control inputs, clock and load. For a rising edge triggered register, the inputs are only stored when load is high and clock is rising from 0 to 1. Another control input clear may be used to resets all bits to 0 regardless of the value of input. Since all n bits of the registers can be stored in parallel, we refer this type of register as a parallel load register.
- A **shift register** stores n bits from its one bit data input with at least two control inputs clock and shift. When clock is rising and shift is 1, the nth bit of input is stored in the (n-1)th bit, and (n-

1)th bit of input is stored in the (n-2)th bit and so on down to the second bit being stored in the first bit. The first bit is shifted out appearing as an output bit. It has one bit output and the input must be shifted into the register serially.

- A counter is a register that adds binary 1 to its stored binary value. In general, a counter has a clear, count and load as a control inputs. Clear resets all stored bits to 0 and a count input enables incrementing on each clock edge. It often has parallel load data input and associated load control signal. A common counter feature is both up and down counting which required an additional control input to indicate the count direction.

A small triangle in the block represents the clock input for any sequential logic. Control inputs in sequential logic can be either synchronous or asynchronous. A synchronous input value only has an effect during a clock edge while an asynchronous input value affects the circuit independent of the clock. Clear control lines are asynchronous inputs while load, shift count control lines are synchronous inputs.

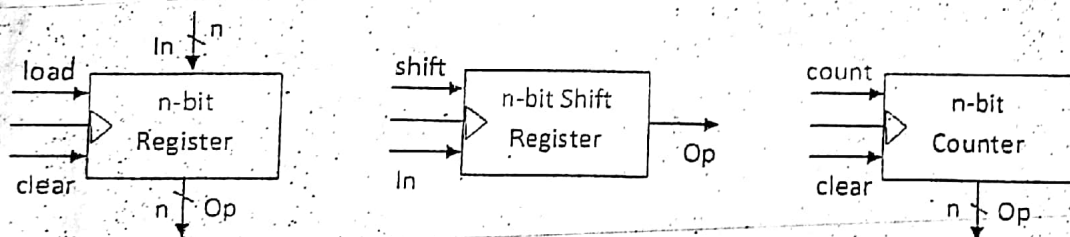


Figure 2.5: RT-Level Sequential Components

Sequential Logic Design

1. Translate the problem description to a state diagram, also called a finite state machine (FSM).
2. In FSM, each circle represents a state where desired output values are listed next to each. Whereas the input conditions which cause a transition from one state to another are listed next to each arc.
3. Draw an implementation model which implements the FSM using a state register to store the current state and combinational logic to generate the required output values and next state.
4. Assign each state a unique binary value, and create a truth table for the combinational logic. The external inputs and the bits coming from the state registers are fed to the combinational logic as inputs. Whereas, the external output values along with the state bits to be loaded into the state register acts as the output of the combinational logic.

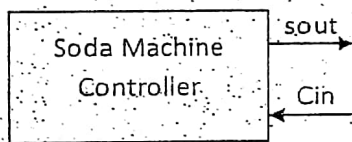
5. The output values change only with the current state, so we list the external output values only for each possible state, regardless of the change in external input values.
6. Now, we can have a truth table, with the help of which we can proceed with combinational design by generating minimized output equations using k-map. And finally, drawing the combinational logic circuit.

Sequential Logic Design Example

Example 1: Design a soda machine controller, given that a soda costs 75 cents and your machine accepts quarters only. Draw a black-box view, come up with a state diagram and state table, minimize the logic, and then draw the final circuit.

Solution: The coin must be entered three times to get a soda out of the machine. Throughout the design, Cin represents the coin input and sout indicates the soda output whereas Q1, Q0 represent current state and I1, I0 represent next state.

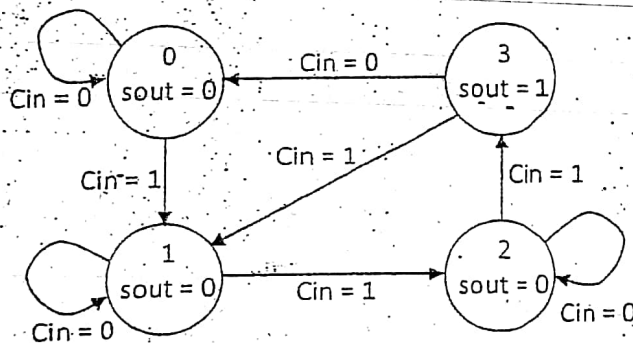
A. Black Box View



C. State Table

Inputs			Outputs		
Q1	Q0	Cin	I1	I0	Sout
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	0	1	1

B. State Diagram



D. K-map

I1

	Q1Q0			
	00	01	11	10
Cin 0	0	0	0	1
1	0	1	0	1

$$I1 = Q1Q0' + Q1'Q0Cin$$

I0

	Q1Q0			
	00	01	11	10
Cin 0	0	1	0	0
1	1	0	1	1

$$I0 = Q1'Q0Cin' + Q1Cin + Q0'Cin$$

sout

	Q1Q0			
	00	01	11	10
Cin 0	0	0	1	0
1	0	0	1	0

$$sout = Q1Q0$$

E. Combinational Circuit

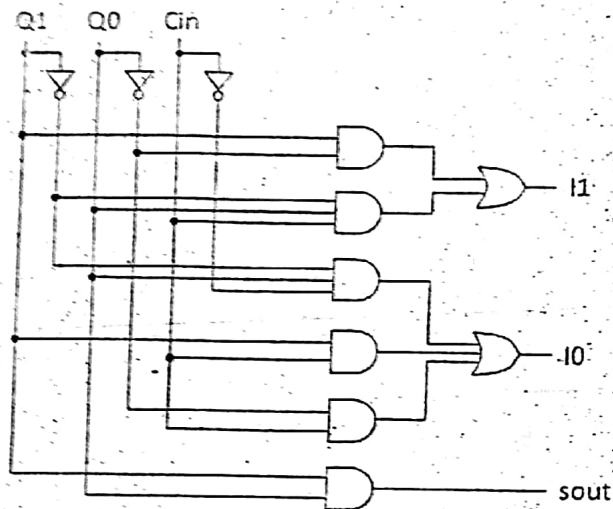


Figure 2.6: Soda machine controller design

2.3 Custom Single-Purpose Processor Design

A basic processor consists of a controller and a datapath.

Datapath

- It stores and manipulates a system's data.
- It contains register units, functional units and connection units like wires & multiplexors.
- The datapath can be configured to read data from particular registers, feed that data through functional units configured to carry out particular operations like add or shift, and store the operation results back into particular registers.
- Examples of data include binary numbers representing external conditions like temperature or speed, characters to be displayed on a screen.

Controller

- It sets the datapath control inputs, like register load and multiplexor select signals, of the register units, functional units, and connection units to obtain the desired configuration at a particular time.
- It monitors external control inputs as well as datapath control outputs, known as status signals, coming from functional units, and it sets external control outputs as well.

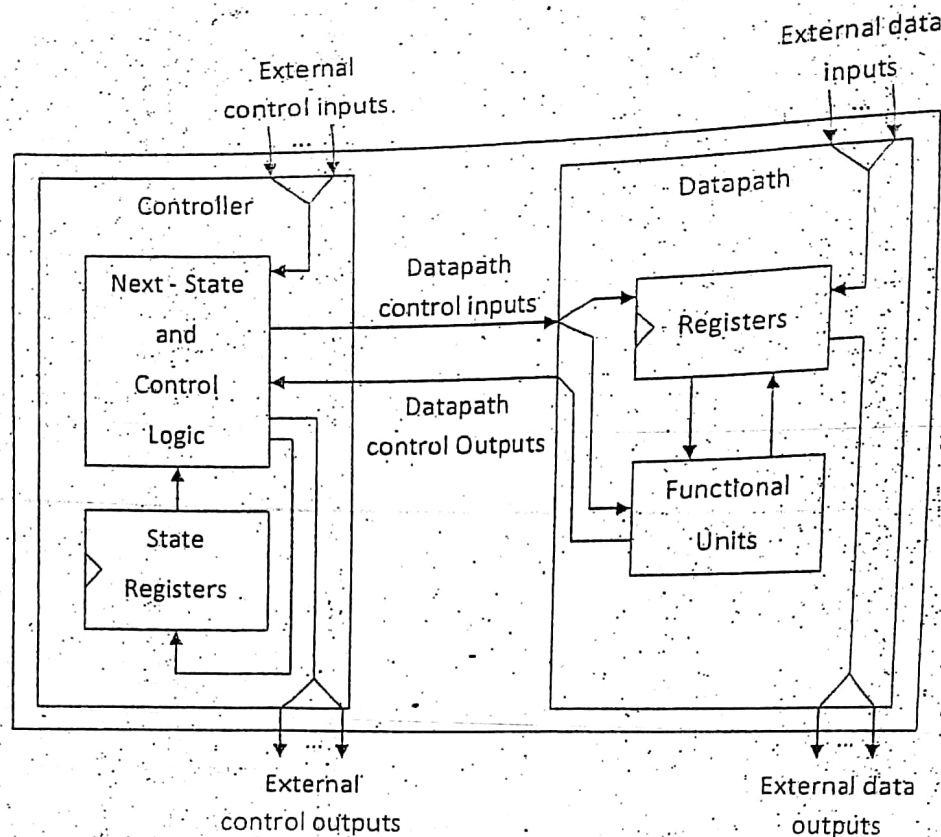


Figure 2.7: Internal View of controller and datapath of Single Purpose Processor.

Steps for designing Single-Purpose Processor

1. **Draw a Black Box Diagram:** Black box diagram is a simple box with external interfaces of a system. It generally includes input and output signals along with few control signals.
2. **Write the functionality or program:** The functionality or program is a code which provides the solution to the defined problem.
 - The input signals are assigned to a variable.
 - Number of temporary variables may be used based on requirement.
 - The final result is assigned to the output port.
3. **Design a Finite State Machine with Data (FSMD):** The code is converted into equivalent complex state diagram which is known as Finite State Machine with Data. In FSMD, Templates are used to represent various constructs of program. The templates for assignment, branch statement and loop statement are discussed below.

- **Assignment Statement:** For this statement, a single state is used with statement representing its action. Generally, a single arrow is used to connect to next state. The template used for statement $C = A + B$ is shown as an example.

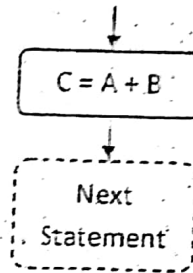


Figure 2.8: Template for assignment statement

- **Branch Statement:** It can be represented by using condition state C, join state J, and few other states in between C and J state. State C and State J are with no actions, left empty. But states between C state and J state contain actions. Its template can vary depending on number of conditions defined in the problem. However, for each true condition, there can be several states representing actions. Conditions are written along side with the arrow that connects the C state and states of each branch. Last states of each branch are connected to the J state.

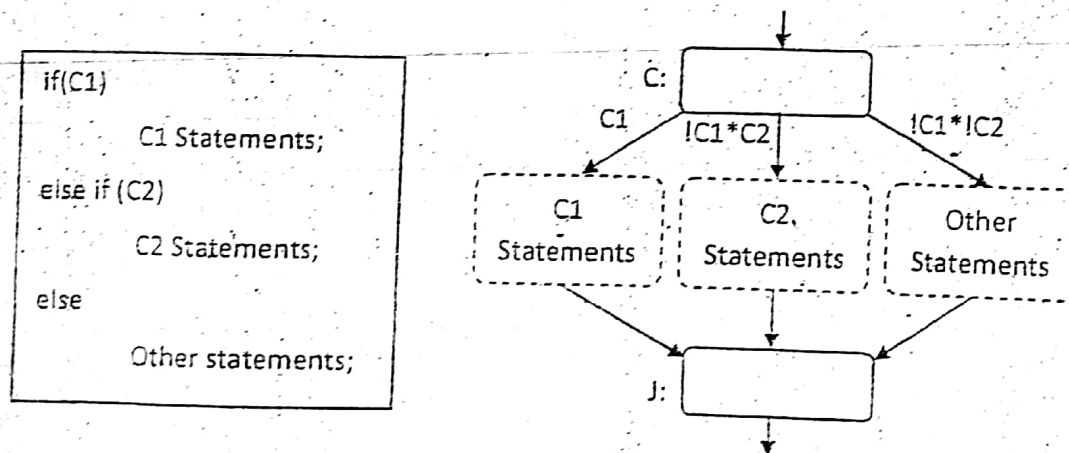


Figure 2.9: Template for branch statement

- **Loop Statement:** Its template consists of Condition State C, Join State J, and other states representing statements of loop. Condition is written alongside arrow connecting condition state and state of first statement of loop. The last state of loop is connected to the J state which is connected back to condition state. Complement condition is used

alongside arrow connecting C state and next statement outside of loop. The template for the loop statement is shown in the figure below.

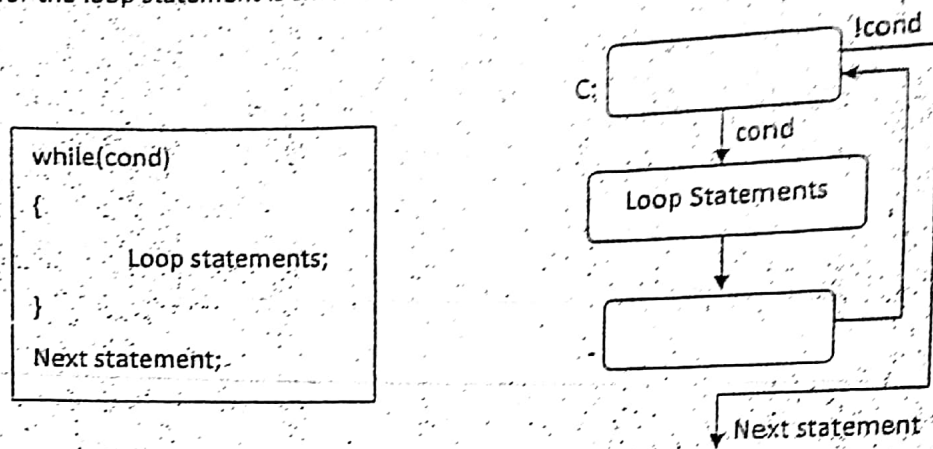


Figure 2.10: Template for loop statement

4. **Build a Datapath:** The datapath is build based on functionality of the system. Following steps are needed to be taken into considerations while developing a datapath.

- **Registers:** The number of registers to be used is defined by the number of variables used in the functionality. Registers are assigned to inputs, temporary variables and output.
- **Functional Units:** Blocks representing arithmetic and logical operations are defined within the datapath.
- **Connections:** The connections among ports, registers and functional units are done based on operands used in various assignments and comparison of functionality code. Appropriate multiplexor is required when the value in register can be assigned from more than one source. The sources may be an input port, a functional unit, or another register.
- **Control inputs and outputs:** Input control signals are generally required by registers and multiplexor. Register load signal is used in case of register while selection line signals for multiplexor. Control output is produced by logical units of the datapath. Each control singles are given a unique identifier.

5. **Develop a Finite State Machine (FSM):** The states and transitions for FSM are same as that of FSMD. However, the complex actions and conditions of FSMD are replaced by Boolean expressions using the control signals defined within datapath. For every register write operations (assignment statement, arithmetic statements), register load signal is asserted and

corresponding multiplexor selection line is activated if there are two or more sources for a given register. Also the logical operations are replaced by the control signals of its corresponding functional block.

Example 1: Design a single purpose processor that calculates the Greatest Common Divisor (GCD) of two numbers. Include FSMD, Datapath and FSM in the design.

→ Initially, the black box view diagram is drawn and then followed by the functionality which is converted into FSMD using appropriate templates.

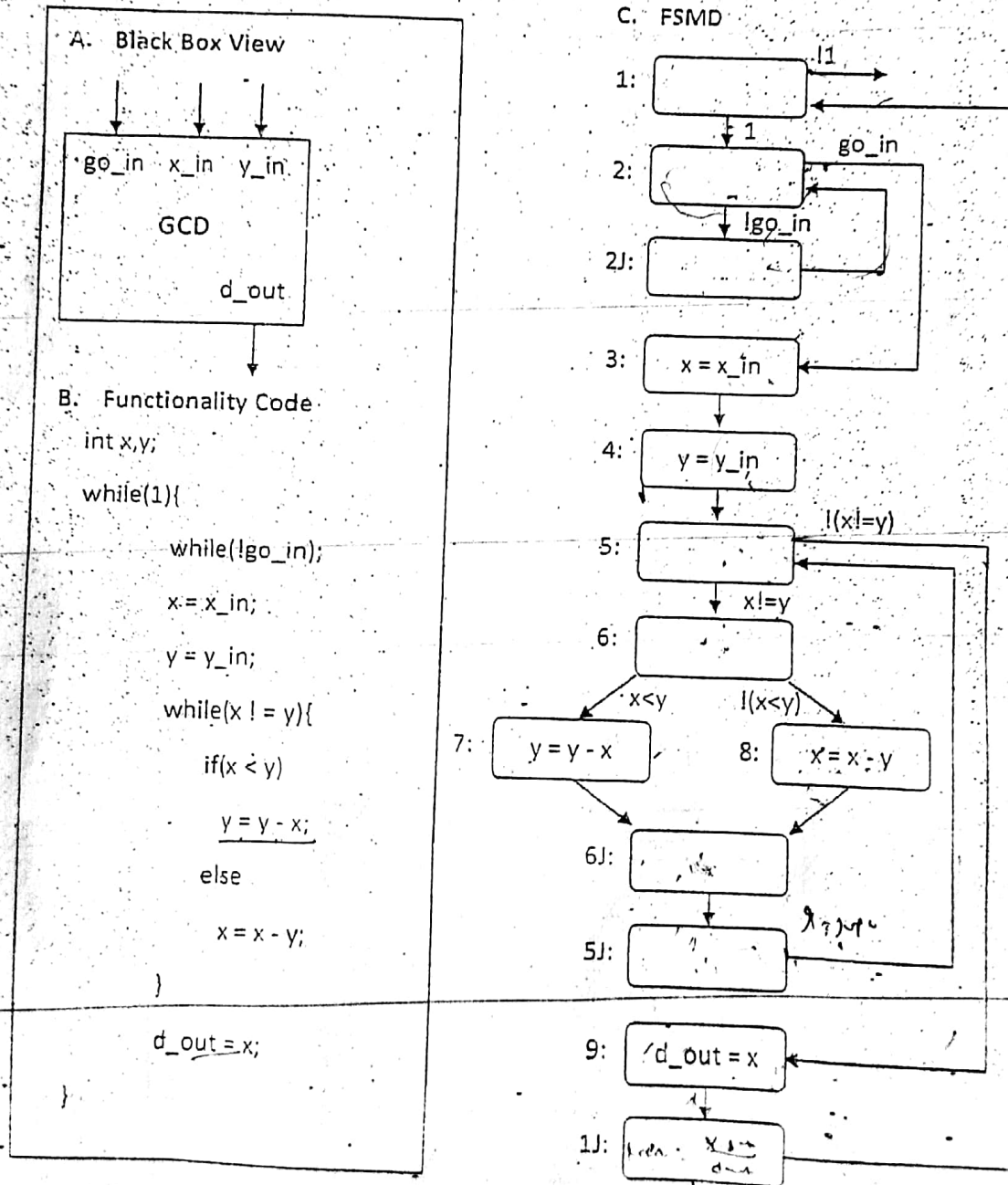


Figure 2.11: Black box view, functionality and FSMD diagram of GCD processor

D. Datapath for GCD processor:

- **Number of Registers:** Two inputs x_{in} and y_{in} assigned to variables x and y , final result assigned to d_{out} , and no other temporary variables are used. Hence, three registers x , y and d are required.
- **Functional Blocks:** The arithmetic and logical operation involved in the functionality are $x-y$, $y-x$, $x=y$ and $x < y$. Hence, two subtractors and two comparing blocks are required.
- **Connections and MUX requirement:** The value in register x has two sources, x_{in} and $x-y$, so it requires a multiplexor of 2×1 . Similar is the case for register y . For connections, the output of registers x and y are connected to inputs of subtracting blocks and comparing blocks. Also, the line representing x_{in} and $x-y$ are connected to the inputs of mux whose output is fed to register x . Similarly, y_{in} and $y-x$ are connected to the register y through mux. And, the output of x register is connected to input of register d . All connections must be done so as to represent the corresponding operation in the functionality.
- **Control Signals:** Unique identifier for various control signals is assigned.
 - o Load signal of registers: x_{ld} for register x , y_{ld} for register y and d_{ld} for register d .
 - o Selection lines of multiplexor: x_{sel} for multiplexor associated with register x and y_{sel} for multiplexor associated with register y .
 - o Signals from logical block: x_{neq_y} and x_{lt_y} are used for x not equal to y and x less than y respectively.

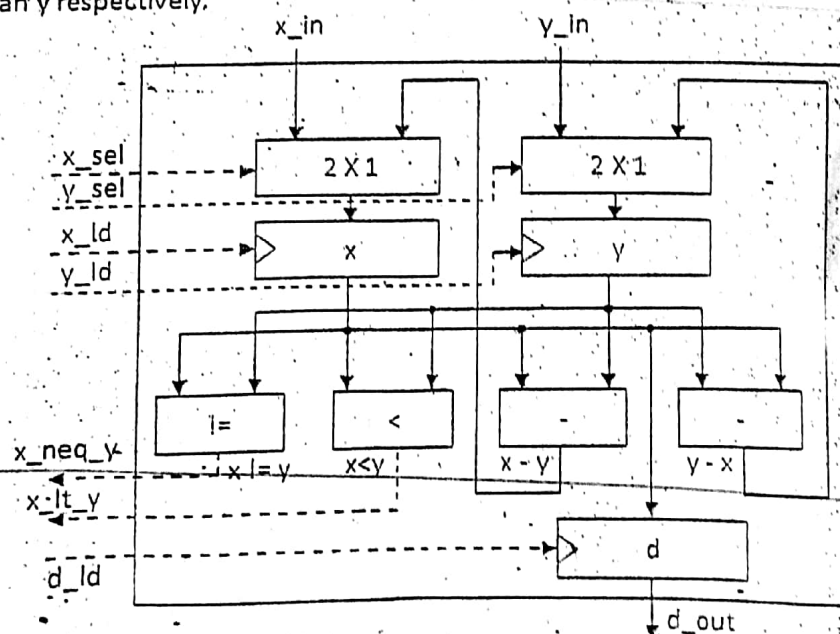


Figure 2.12: Datapath of GCD Processor

E. Finite State Machine for GCD processor

All actions and conditions are replaced by equivalent Boolean expressions as used in datapath. For example, action $x = x_{in}$ is replaced by expressions $x_{sel} = 0$ and $x_{ld} = 1$. $x_{sel} = 0$ will connect the input line x_{in} to register x and $x_{ld} = 1$ will load the value of x_{in} into x . In case of $d_{out} = x$, only $d_{ld} = 1$ is used as it has only one source and no multiplexor is used. And condition $x < y$ is replaced by x_{lt}_y . The identifiers for control signals, however, used in FSMD must match with the one that is defined in datapath.

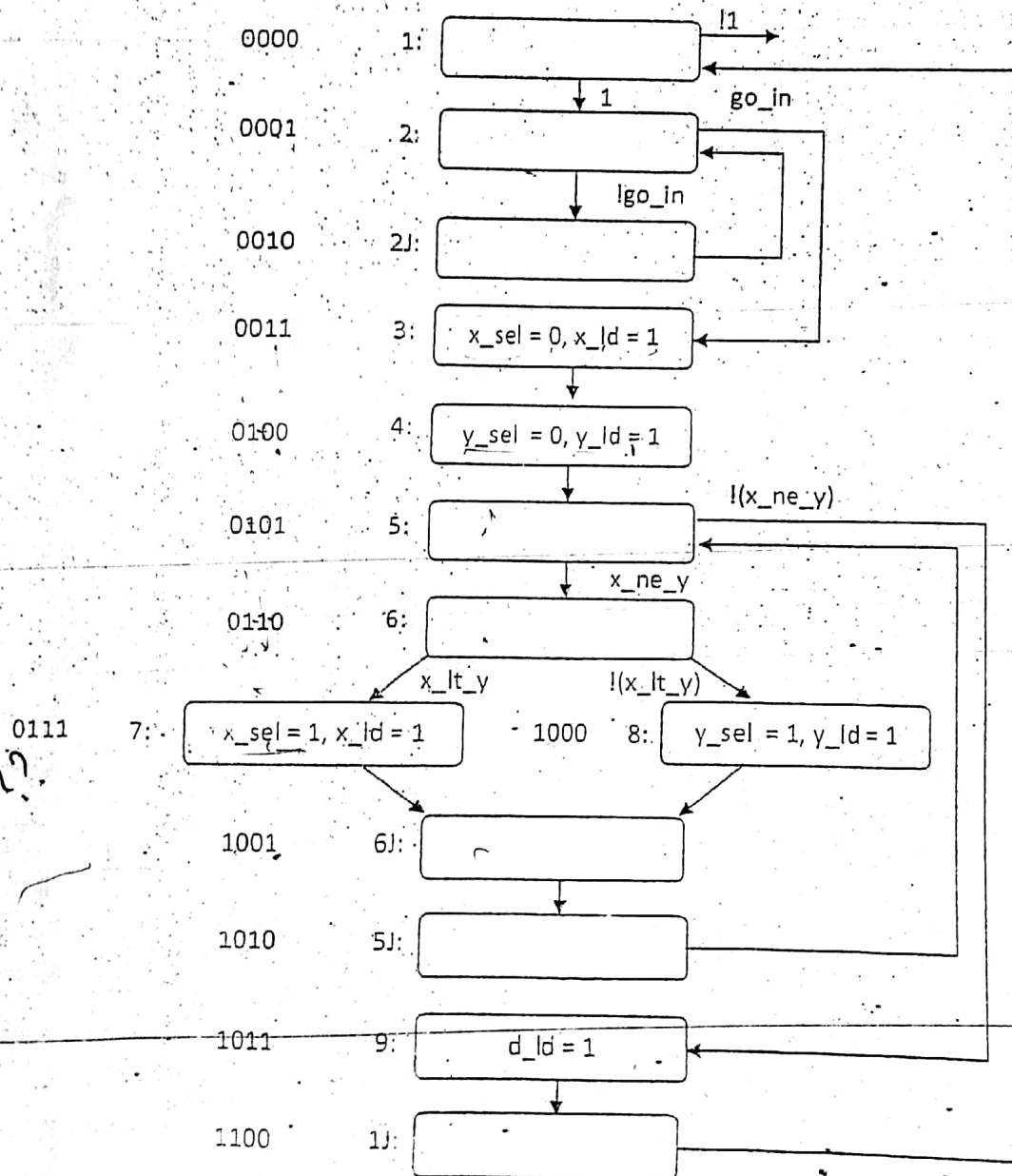


Figure 2.13: FSM of GCD Processor

2.4 Optimizing Custom Single-Purpose Processors

Optimization is the task of making design metric values the best possible. Optimization can be done by simplifying the resulting design of any system utilizing various techniques. Different states in the FSM can be removed which does nothing and are redundant. Also, we can share a component for same operations in different states and hence minimizing the size of the system as well as its cost. Other various factors can be considered for optimum design but some simple optimization that can be applied are discussed further.

Optimizing the Original Program

We should analyze different program attributes and try to develop alternative algorithm that are more efficient. We can analyze the algorithm in terms of time complexity and space complexity. Number of computations can be a form of time complexity whereas the size of variables required corresponds space complexity.

Lets take the example of GCD:

```
int x,y;
while(1){
    while(!go_in);
    x = x_in;
    y = y_in;
    while(x != y){
        if(x < y)
            y = y-x;
        else
            x = x-y;
    }
    d_out = x;
}
```

```
int x,y,r;
while(1){
    while(!go_in);
    x = x_in;
    y = y_in;
    while(y != 0){
        r = x % y;
        x = y;
        y = r;
    }
    d_out = x;
}
```

To compute GCD of 42 and 8, it takes 9 iterations to complete the operation, x and y will take different values as (42, 8), (34, 8), (26, 8), (18, 8), (10, 8), (2, 8), (2, 6), (2, 4), (2, 2).

To compute GCD of 42 and 8, it takes 3 iterations to complete the operation, x and y will take values as (42, 8), (8, 2), (2, 0). If y is greater than x, it will take 4 iterations, one more than previous.

Optimizing the FSMD

Each state in an FSMD is assigned with operations from the desired program; this process is also termed as scheduling. The scheduling process can be improved by following methods.

- **Merge States:** States with independent operations can be merged.
- **Eliminate State:** States with constants on transitions can be eliminated since transition to be taken will be fixed as defined by constants. And some states without any operation can also be eliminated.
- **Separate States:** States which require complex operations can be broken into smaller states to reduce hardware size.

Considering the example of GCD:

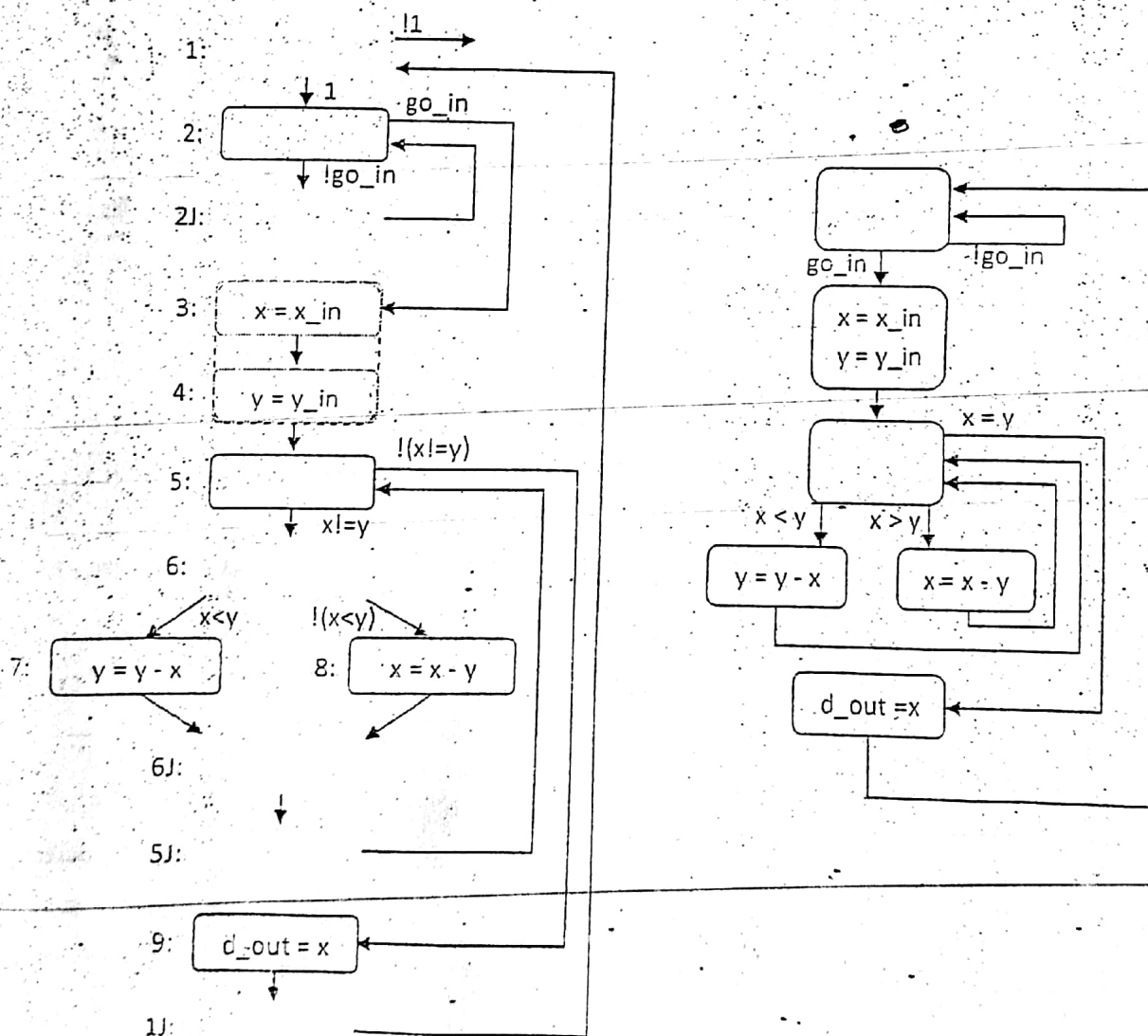


Figure 2.14: Optimized FSMD from original FSMD

The following actions are taken to optimize the original FSM.

- Eliminate state 1 – transitions have constant values
- Merge state 2 and 2J – loop has no body
- Merge state 3 and 4 – operations are independent of each other
- Merge state 5 and 6 – transitions from state 6 can be done in state 5
- Eliminate state 5J and 6J – transitions from each state can be done from state 7 and 8 respectively
- Eliminate state 1J – transitions from state 1J can be done directly from state 9

Consider the operation $p = a * b * c * d$; if we use single state for this particular operation then three multipliers are required which renders system expensive and bulky. So the operation can be broken down as $x = a * b$, $y = c * d$ and $p = x * y$ with each operations having its own state. Thus, only one multiplier would be required in the system.

Optimizing Datapath

During the datapath design, the task of selecting a RT components for particular operation is termed as allocation. Whereas the task of mapping operations from the FSM to allocated components is termed as binding. The optimization in datapath design can be done by following ways.

- **Sharing of Functional Units:** Single functional unit can be shared if same operations occur in different states. For example; in computation of GCD there were two subtractor used for two subtraction, rather a single subtractor can be used with the help of the multiplexor. Hence one to one mapping is not necessary.
- **Use of Multi-functional Units:** A variety of operations can be performed by ALU hence it can be shared for different operations occurring in different states.

Optimizing the FSM

Optimization in FSM can be done by:

- **State Encoding:** It is the task of assigning a unique bit pattern to each state in an FSM. The size of the register as well as the size of the combinational logic varies for different encodings. For example, if we have four states then it can be encoded as 00, 01, 10, 11 but it can also be encoded as 11, 10, 01, 00. If the number of state is large the number of ways of state encoding will be very large, hence CAD tools are used to determine the most efficient encodings.

- **State Minimization:** It is the task of merging equivalent states into a single state. Two states are equivalent if those two states generate the same outputs and transition to the same next state, for any given input combinations. Merging equivalent states yield exactly the same output behaviour.

Few Solved Examples

Problem 1: Design a combinational logic circuit for the given problem whose description is given as: y is 1 if a is 1, or b and c are 1. z is 1 if b or c is 1, but not both (or, a , b , and c are 1).

Solution: Initially, truth table is formed by writing down all possibilities of inputs followed by writing the outputs as defined by the given problem. Then, the K-map is used to minimize the equations and finally the combinational circuit is drawn.

A. Truth table

Inputs			Outputs	
a	b	c	y	z
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

B. K-map

y	bc			
	00	01	11	10
a				
0	0	0	1	0
1	1	1	1	1

$y = a + bc$

z	bc			
	00	01	11	10
a				
0	0	1	0	1
1	0	1	1	1

$z = ab + b'c + bc'$

C. Combinational Circuit

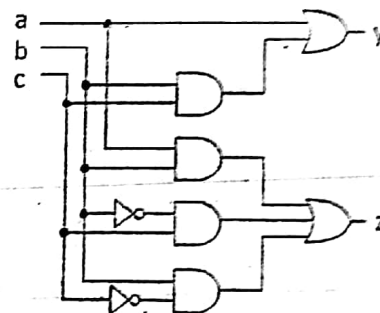


Figure 2.15: Truth table, K-map, and combinational circuit

Problem 2: Design a 2-bit comparator with a single output "less than", using the combinational design technique described in the chapter. Start from a truth table, use K-maps to minimize logic and draw the final circuit.

Solution: As the comparator is 2 – bit, there must be total of four inputs; two inputs each of two bits. And only less than condition is to be checked, so only single output must be defined. Then the general steps for designing a combinational logic circuit is followed.

A. Truth table

a_1	a_0	b_1	b_0	It
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

B. K - map

		a_1a_0			
		00	01	11	10
b_1b_0	00	0	0	0	0
	01	1	0	0	0
	11	1	1	0	1
	10	1	1	0	0

$$It = b_1a_1' + b_0a_1'a_0' + b_1b_0a_0'$$

C. Combinational Circuit

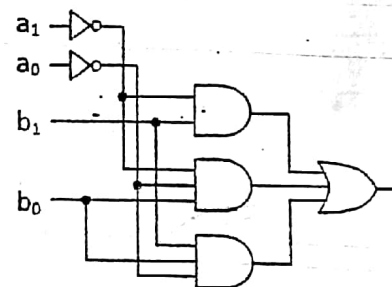


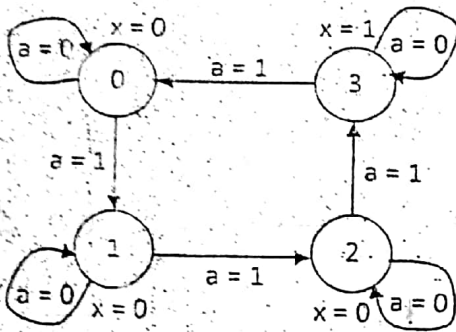
Figure 2.16: Truth table, K-map and combinational circuit for two bit comparator

Handwritten notes:
 20
 20/11/2017
 20/11/2017
 20/11/2017

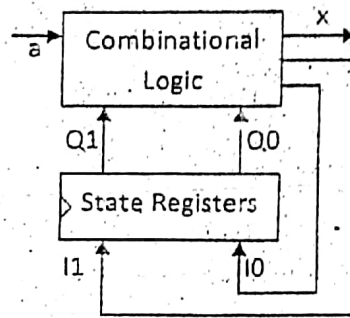
Problem 3: Construct a pulse divider. Slow down your pre-existing pulse so that you output a 1 every four pulses detected.

Solution:

A. State Diagram



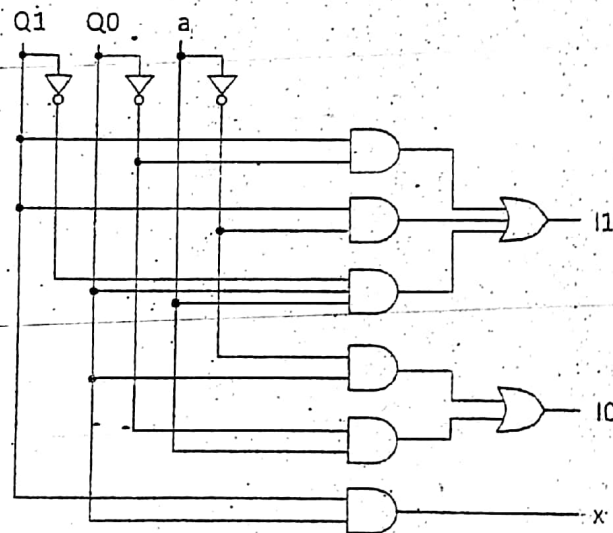
B. Implementation Model



C. State Table

Inputs			Outputs		
Q1	Q0	a	I1	I0	x
0	0	0	0	0	0
0	0	1	0	1	
0	1	0	0	1	0
0	1	1	1	0	
1	0	0	1	0	0
1	0	1	1	1	
1	1	0	1	1	1
1	1	1	0	1	

E. Combinational Circuit



D. K-map

I1	Q1Q0			
	00	01	11	10
a 0	0	0	1	1
1	0	1	0	1

$$I1 = Q1'Q0a + Q1a' + Q1Q0'$$

I0	Q1Q0			
	00	01	11	10
a 0	0	1	1	0
1	1	0	0	1

$$I0 = Q0a' + Q0'a$$

x	Q1Q0			
	00	01	11	10
a 0	0	0	1	0
1	0	0	1	0

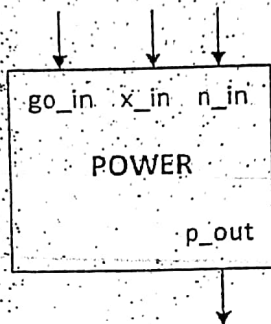
$$x = Q1Q0$$

Figure 2.17: Pulse Divider – State diagram, state table, K-map, combinational circuit

Problem 4: Design a single purpose processor that calculates x to the power n (x^n). Include FSMD, Datapath and FSM in the design.

Solution:

A. Black Box View



B. Functionality Code

```
int x,n,p;
while(1){
    while(!go_in);
    x = x_in;
    n = n_in;
    m = 1;
    while(n>0){
        m = m * x;
        n = n - 1;
    }
    p_out = m;
}
```

C. FSMD

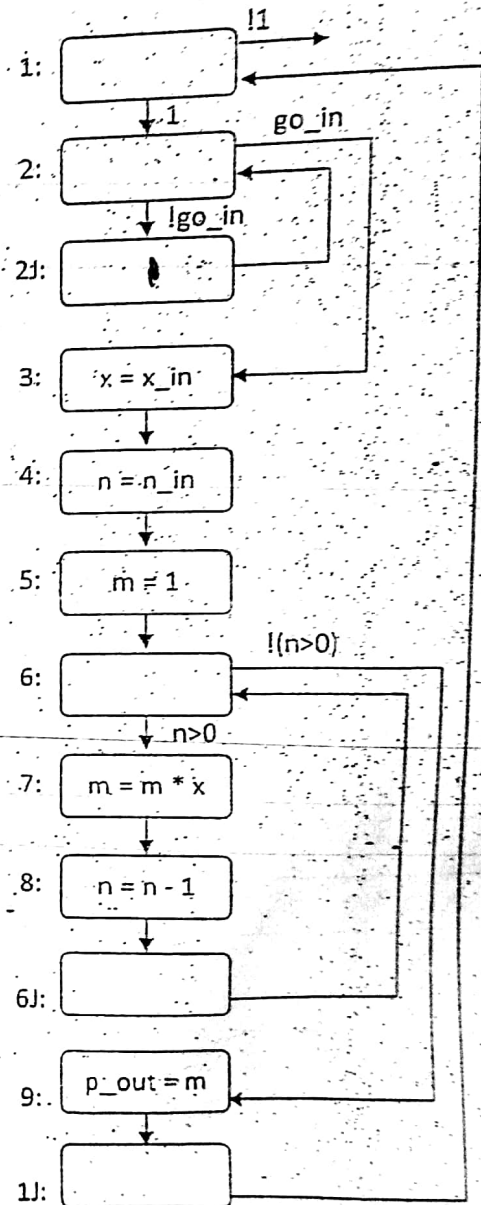


Figure 2.18: The black box view, functionality and FSMD for processor that calculates x^n .

D. Datapath for the processor that calculates x^n

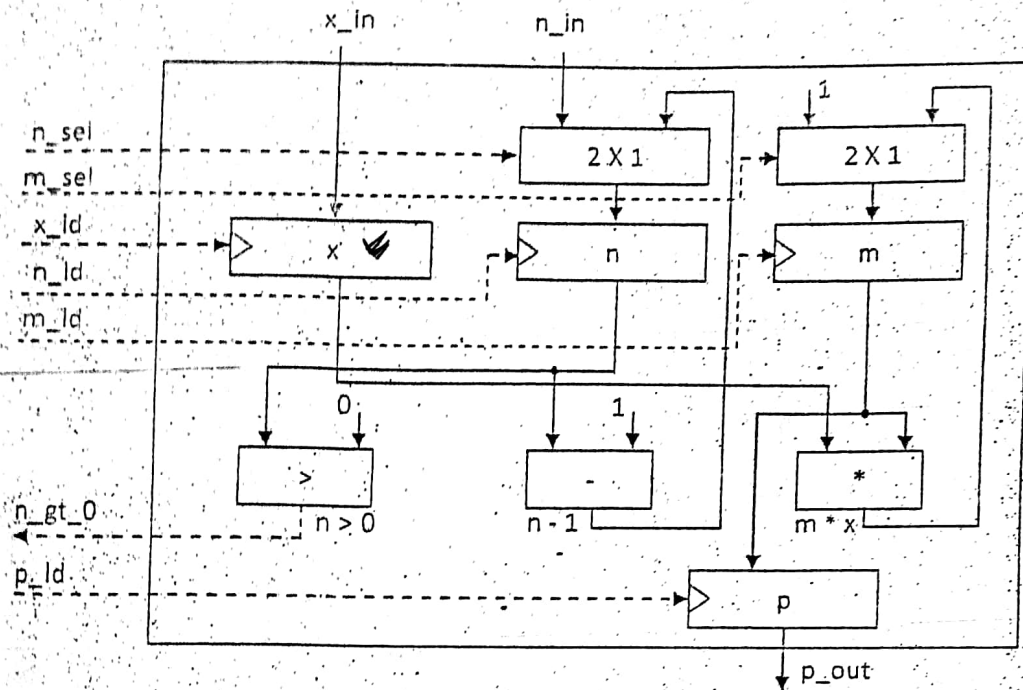


Figure 2.19: Datapath

E. FSM of the processor that calculates x to the power n .

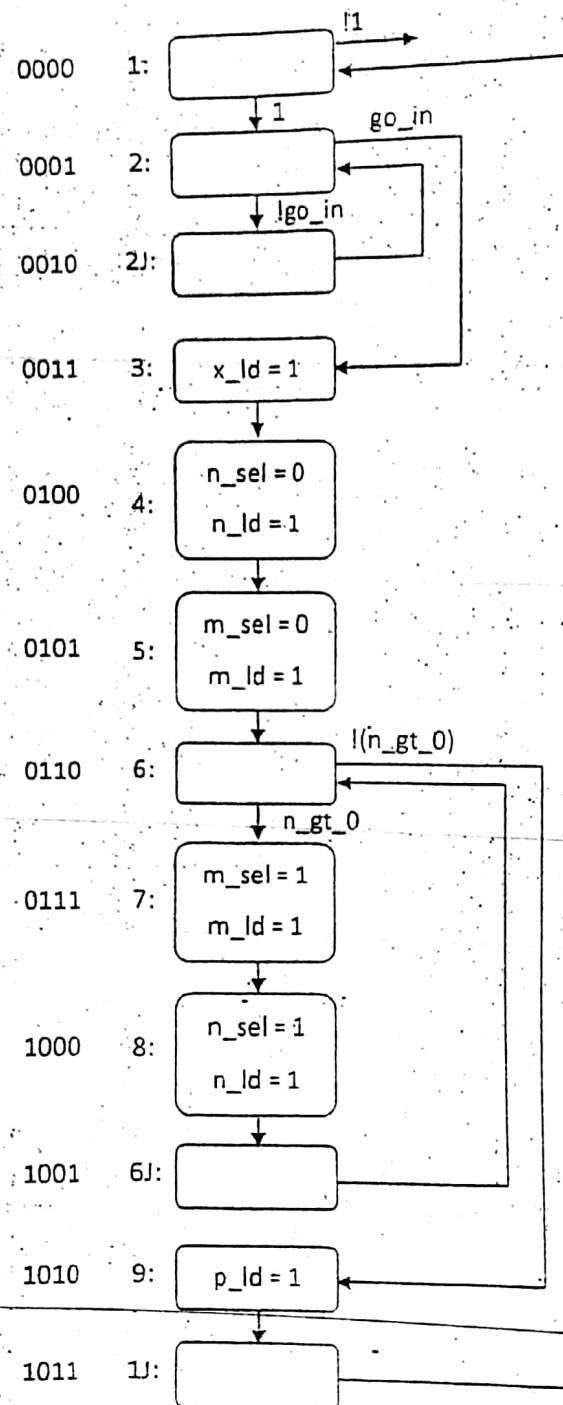
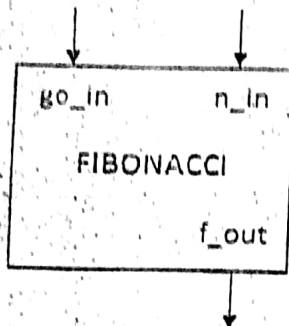


Figure 2.20: FSM Controller

Problem 5: Design a single purpose processor that generates Fibonacci series up to n places. Start with a function that computes desired result, translate the function into a state diagram, sketch a probable datapath, and draw FSM diagram.

Solution:

A. Black Box View



B. Functionality Code

```

int ft, st, nt, count, n;
while(1){
    while(!go_in);
    n = n_in;
    ft = 0;
    st = 1;
    count = 1;
    while(count <= n){
        f_out = ft;
        nt = ft + st;
        ft = st;
        st = nt;
        count++;
    }
}
  
```

C. FSMD

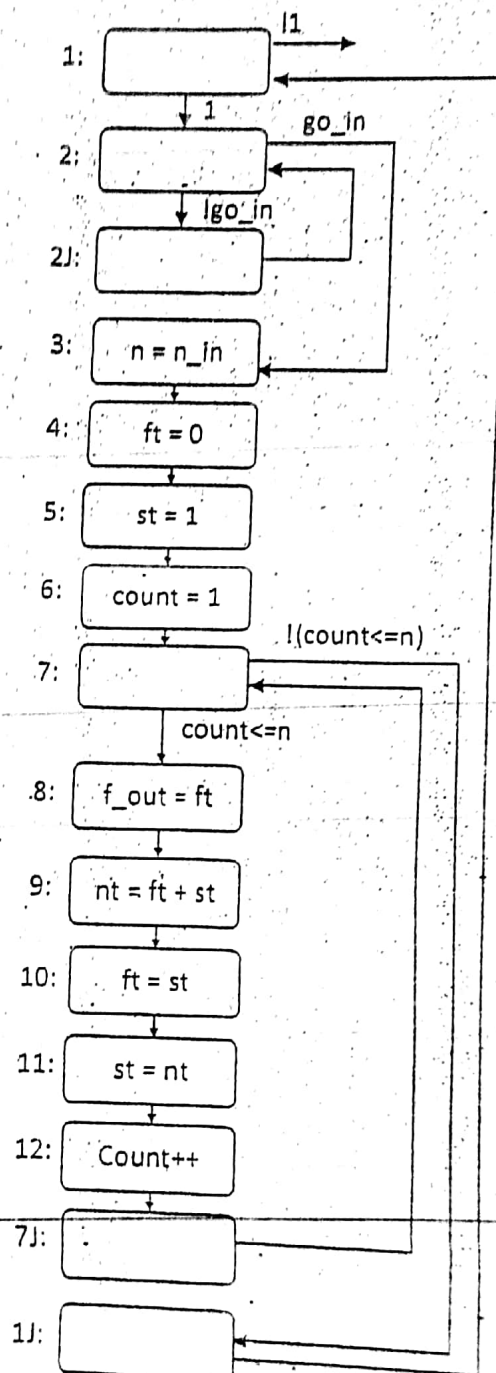


Figure 2:21: Fibonacci series generator – the black box view, functionality and FSMD

D. Datapath of the processor that generates Fibonacci series:

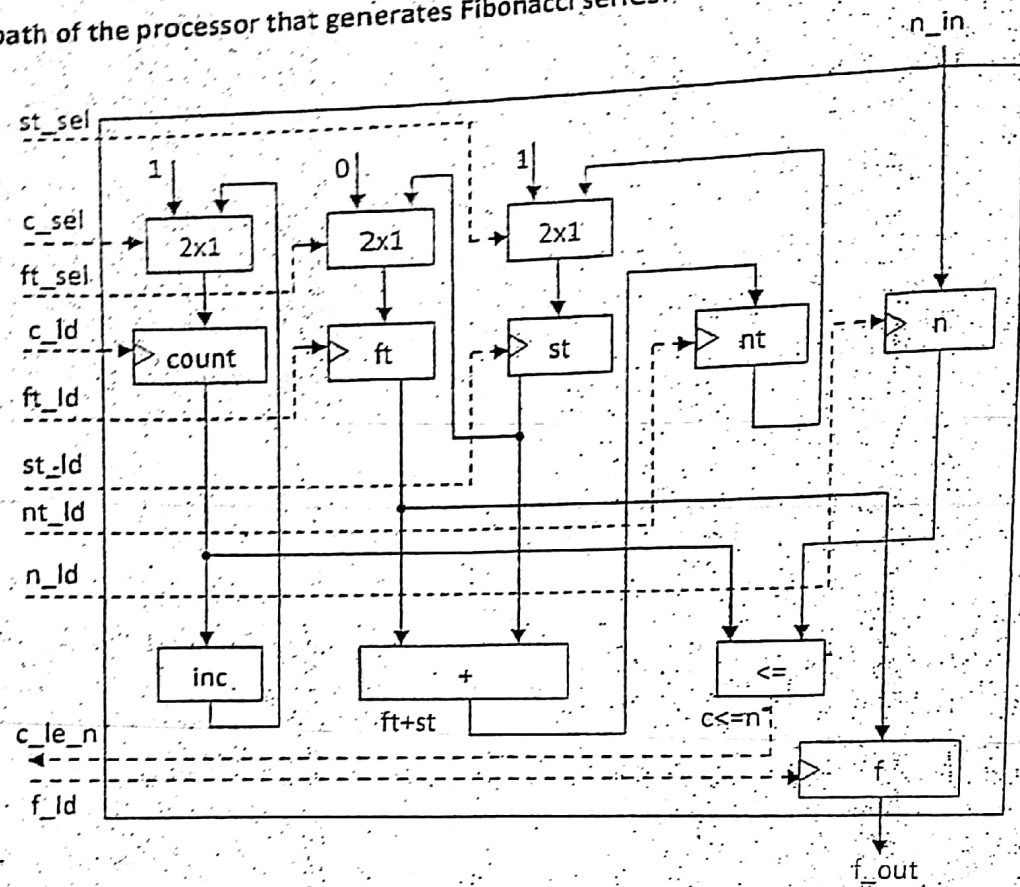


Figure 2,22: Datapath for Fibonacci series generator

E. FSM controller for Fibonacci series generator

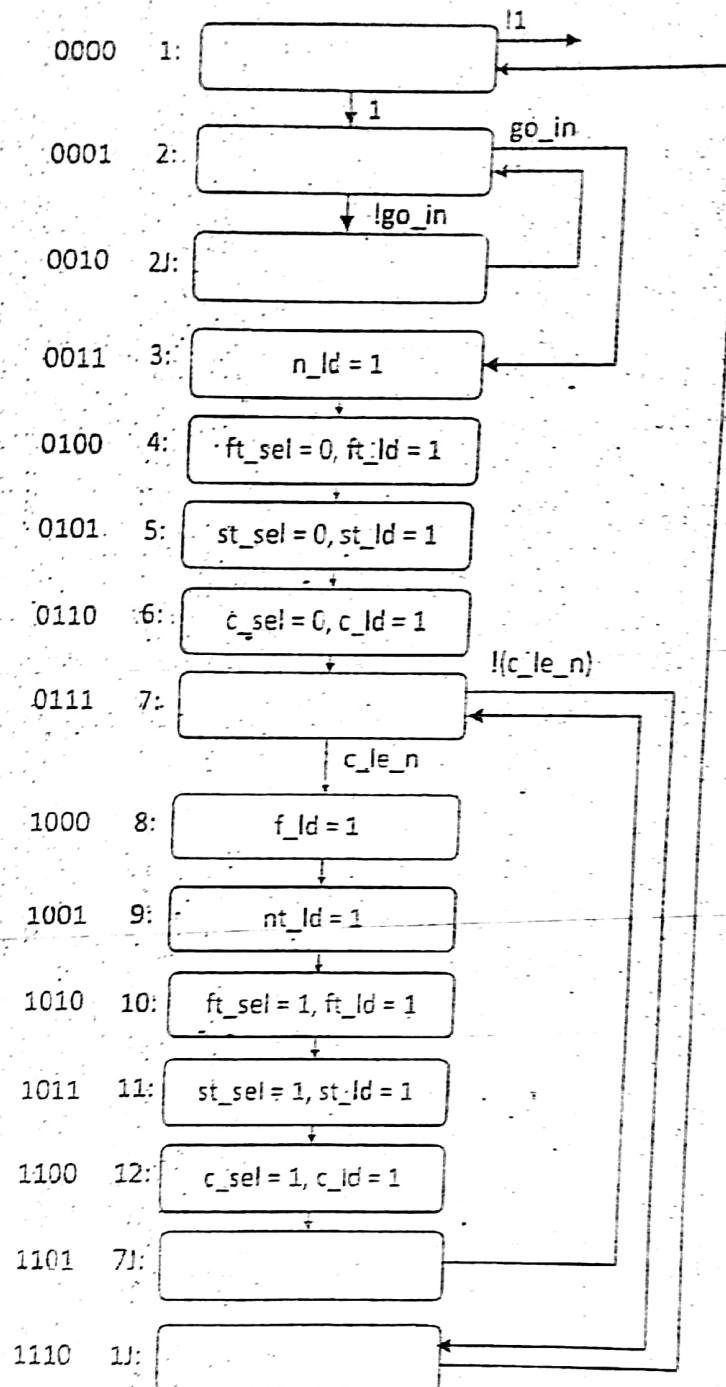


Figure 2.23: FSM of Fibonacci series generator